Basic commands

uname

- Display system information.
  - Example:

```
Command :$ uname -a
Output  :Linux hostname 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10
00:24:02 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
```

- `uname -r` shows the kernel version.
- `uname -m` shows the machine hardware name (e.g., x86_64).
- `uname -s` shows the operating system name (e.g., Linux).

pwd

- Print the current working directory.
- Shows the full path of the directory you are currently in.
  - Example:

```
Command: $ pwd
Output : /home/user/documents
```

- It shows the absolute path from the root directory to your current location in the filesystem.

ls

- List files and directories in the current directory.
  - Example:

```
Command :$ ls
Output  :file1.txt  file2.txt  directory1  directory2
```

- Lists the names of files and directories in the current directory.
- By default, it does not show hidden files (those starting with a dot).
- Commonly used options with `ls`:
  - ls -l
    - Lists files and directories in long format, providing detailed information.
  - ls -a
    - Lists all files, including hidden files.

`

    - ls -S
        - Sorts files by size, with the largest files listed first.
    - ls -t
        - Sorts files by modification time, with the most recently modified files listed first.
    - ls -R
        - Recursively lists files in all subdirectories.
    - ls -lh
        - Lists files in long format with human-readable file sizes (e.g., KB, MB).
    - ls -la
        - Combines the -l and -a options to show detailed information for all files, including hidden ones.
- Combining Options:
    - You can combine multiple options together.
    - for more details use `man ls` command.

dir

- List files and directories in a directory.
    - Example:

```
Command :$ dir
Output  :file1.txt  file2.txt  directory1  directory2
```

- Similar to `ls`, it lists the names of files and directories in the current directory.
- By default, it does not show hidden files (those starting with a dot).

tree

- Display a tree-like structure of directories and files.
    - Example:

```
Command :$ tree
Output  :.
          ├── directory1
          │   ├── file1.txt
          │   └── file2.txt
          ├── directory2
          │   └── file3.txt
          └── file4.txt
```

- Shows the hierarchical structure of directories and files in a tree format.
- By default, it displays the current directory and its contents recursively.

cd

- Change the current directory.

    - Example:

```
Command :$ cd /home/user/documents
```

- Changes the current working directory to the specified path.
- You can use relative paths (e.g., `cd ../` to go up one directory)
- Or absolute paths (e.g., `cd /home/user/documents`).
- Shell Metacharacters:
    - `/` : Represents the root directory.
      - For example, `cd /` takes you to the root of the filesystem.
    - `~` : Represents the home directory of the current user.
      - For example, `cd ~` takes you to your home directory.
    - `.` : Represents the current directory.
      - For example, `cd .` keeps you in the same directory.
    - `..` : Represents the parent directory (one level up).
      - For example, `cd ..` takes you to the parent directory of your current location.
    - `-` : Represents the previous directory you were in.
      - For example, `cd -` switches back to the last directory you were in.

mkdir

- Create a new directory.
    - Example:

```
Command :$ mkdir new_directory
or
Command :$ mkdir -p parent_directory/child_directory
```

- Creates a new directory with the specified name.
- The `-p` option allows you to create parent directories as needed.
- If the parent directories do not exist, they will be created automatically.
- If the directory already exists, an error message will be displayed unless you use the `-p` option.

rmdir

- remove an empty directory.
    - Example:

```
Command :$ rmdir empty_directory_name
or
Command :$ rmdir -p parent_directory/child_directory
```

- Removes the specified empty directory.
- The `-p` option allows you to remove parent directories as long as they are empty.

- If the directory is not empty, an error message will be displayed.

touch

- Create a new empty file or update the timestamp of an existing file.
- Example:
  `bash Command :$ touch newfile.txt`
- Creates a new empty file with the specified name if it does not already exist.
- If the file already exists, it updates the file's access and modification timestamps to the current time.

rm

- Remove files or directories.
  - Example:

```
Command :$ rm file.txt
or
Command :$ rm -r directory_name
```

- Removes the specified file.
- The `-r` option allows you to remove directories and their contents recursively.
- Be cautious when using `rm`, as it permanently deletes files and directories without moving them to a trash or recycle bin.

`

`

`

cp

- Copy files or directories.
  - Example:

```
Command :$ cp source_file.txt destination_file.txt
or
Command :$ cp -r source_directory/ destination_directory/
```

- Copies the specified source file to the destination file.
- The `-r` option allows you to copy directories and their contents recursively.
- If the destination file already exists, it will be overwritten without warning.

mv

- Move or rename files or directories.
  - Example:

```
Command :$ mv old_file.txt new_file.txt
or
Command :$ mv file.txt /path/to/destination/
```

- Moves the specified file to a new location or renames it.
- If the destination is a directory, the file will be moved into that directory.
- If the destination file already exists, it will be overwritten without warning.

cat

- Concatenate and display the contents of files.
  - Example:

```
Command :$ cat file.txt
```

- Displays the contents of the specified file in the terminal.
- You can also use `cat` to concatenate multiple files and display their combined contents.
- For example, `cat file1.txt file2.txt` will display the contents of both files sequentially.

```
Command :$ cat -n file.txt
Output  :Displays the contents of the file with line numbers.
```

- cat can also be used to create new files or append text to existing files using redirection operators (`>` and `>>`).
  - For example, `cat > newfile.txt`
    - allows you to type text that will be saved to `newfile.txt` when you press `Ctrl + D`.
  - `cat >> existingfile.txt` allows you to append text to `existingfile.txt`.

echo

- Display a line of text or a variable's value.
  - Example:

```
Command :$ echo "Hello, World!"
Output  :Hello, World!
```

- Prints the specified text or the value of a variable to the terminal.
- You can use it to display messages, concatenate strings, or show the values of environment variables.

which

- Locate the executable file associated with a given command

    - Example:

```
Command :$ which python
Output  :/usr/bin/python
or
Command :$ which ls
Output  :/bin/ls
```

- It searches the directories listed in the PATH environment variable and returns the full path of the executable file for the specified command.
- If the command is not found, it will not return any output.

whoami

- Display the current user's username
    - Example:

```
Command :$ whoami
Output  :user
```

- Shows the username of the currently logged-in user.

`

`

`

`

date

- Display or set the system date and time.
    - Example:

```
Command :$ date
Output  :Mon Oct  2 14:30:00 UTC 2023
```

- Displays the current date and time in a human-readable format.

cal

- Display a calendar for a specific month or year.
    - Example:

```
Command :$ cal
Output  :    October 2023
         Su Mo Tu We Th Fr Sa
            1  2  3  4  5  6
          7  8  9 10 11 12 13
         14 15 16 17 18 19 20
         21 22 23 24 25 26 27
         28 29 30 31
```

- Displays the calendar for the current month by default.
- You can specify a month and year to display a different calendar.
    - For example, `cal 12 2023` will show the calendar for December 2023.
    - `cal 2023` will show the calendar for the entire year 2023.
    -

clear

- Clear the terminal screen
    - Example:

```
Command :$ clear
```

- Clears all previous commands and output from the terminal, providing a clean workspace.

`

`

`

`

factor

- Display the prime factorization of a number.
    - Example:

```
Command :$ factor 28
Output  :28: 2 2 7
```

- Shows the prime factors of the specified integer.
- You can input any positive integer, and it will return its prime factors.

sleep

- Pause execution for a specified amount of time.

- Example:

```
Command :$ sleep 5
```

- Pauses the execution of the script or command for the specified number of seconds (in this case, 5 seconds).
- Useful for creating delays in scripts.

history

- Display the command history.
  - Example:

```
Command :$ history
Output  :1  pwd
          2  ls
          3  cd documents
          4  mkdir new_folder
          ...
```

- Running a Previous Command
  - Example:
  - ```
    Command : $!10
    Output  : #Runs command number 10 from history.
    ```

`

`

- Repeat the Last Command
  - Example:
  - ```
    Command : $!!
    Output  : #Repeats the last command.
    ```

- Run the Most Recent Command Starting with 'sleep'
  - Example:

```
Command : $!sleep
Output  : #Runs the most recent command that started with sleep.
```

- Searching Command History
  #Press Ctrl + R and type a keyword to search backward.

- Clearing Command History

  - Example:

```
Command : $history -c
Output  : #Clears the command history.
```

`

`

man

- Display the manual pages for a command.
  - Example:

```
Command : $man command_name
Output  : #Displays the manual page for the specified command.
```

- Provides detailed information about a command, including its options, usage, and examples.
- You can navigate through the manual pages using the arrow keys, Page Up/Page Down, and search functionality (by pressing `/` followed by the search term).
- To exit the manual page, press `q`.
- For example, `man ls` will show the manual page for the `ls` command.
- If a command does not have a manual page, it will display a message indicating that no manual entry is found.
- Manual Sections:
- The manual pages are divided into sections, each covering different types of information. Common sections include:
  1. Executable programs or shell commands
  2. System calls (functions provided by the kernel)
  3. Library calls (functions within program libraries)
  4. Special files (usually found in /dev)
  5. File formats and conventions, e.g., /etc/passwd
  6. Games
  7. Miscellaneous (including macro packages and conventions), e.g., man(7), groff(7)
  8. System administration commands (usually only for root)
  9. Kernel routines [Non-standard]
  - Example

```

```

```
$ man [command_name]
$ man section_number command_name
   - check specific section of the manual.
$ man man: Displays information about the man command.
```

Command Line Keyboard Shortcuts

```
Shortcut  : Description
Ctrl + a  : Jump to the beginning of the command line
Ctrl + e    : Jump to the end of the command line
Ctrl + u    : Clear from the cursor to the beginning of the command line
Ctrl + k    : Clear from the cursor to the end of the command line
Ctrl + Left Arrow   : Jump to the beginning of the previous word on the command
line
Ctrl+Right Arrow  : Jump to the beginning of the next word on the command line
Ctrl + r : Search the history list of commands for a pattern
Ctrl + l : Clear the terminal screen (similar to the clear command).
Ctrl + c : Cancel the current command or process.
Ctrl + d : Log out of the current session or close the terminal (if at the command
prompt).
Ctrl + z  : Suspend the current process and move it to the background.
Tab       : Auto-complete file and directory names.
Up Arrow  : Scroll through previous commands in history.
Down Arrow: Scroll through more recent commands in history.
```
